

# 09 - DTO's

📅 09-09-2026 ⌚ Ongeveer 27 minuten 🗨️ Ongeveer 4018 woorden

In het vorige hoofdstuk zagen we hoe we met `. Include()` en `. ThenInclude()` complexe relaties uit de database konden ophalen. Hierbij liepen we echter al snel tegen het beruchte `Circular Reference` probleem aan. We hebben dat toen tijdelijk opgelost met `ReferenceHandler.IgnoreCycles` in `Program.cs`, maar we gaven toen al aan: dit is een 'quick & dirty' pleister, **geen definitieve oplossing!** Op het examen is deze manier niet voldoende; daar zal je DTO's moeten gebruiken.

## Terugblik & Focus

In dit hoofdstuk stappen we af van het rechtstreeks exporteren of importeren van onze **Domain Models** naar de buitenwereld. We introduceren `Data Transfer Objects (DTO's)`.

Met DTO's lossen we niet alleen het circulaire referentieprobleem definitief op, maar beveiligen we onze API ook tegen het verzenden van data die we niet aan de buitenwereld willen tonen. Daarnaast verkleinen we de netwerkpayload en ontkoppelen we onze interne databasestructuur van de externe API-contracten.

## Wat is een DTO?

Een DTO (`Data Transfer Object`) is een (vaak kleine) C# klasse die **enkel de data bevat die we willen versturen of ontvangen** via een endpoint.

Bekijk onderstaande Modellen. Indien we alle bestellingen van een klant willen tonen, moeten we al deze Modellen ophalen. Dat is erg veel informatie om naar de client te sturen en het wordt al snel onoverzichtelijk. Om nog maar te zwijgen over het circulaire probleem waarmee we in het vorige hoofdstuk geconfronteerd werden.

```
1 public class Bestelling                                     csharp
2 {
3     public int Id { get; set; }
4     public int KlantId { get; set; }
5     public Klant? Klant { get; set; }
6     public List<OrderLijn> OrderLijnen { get; set; }
7 }
```

```
8 public class Klant
9 {
10     public int Id { get; set; }
11     public string Naam { get; set; }
12     public string Voornaam { get; set; }
13     public DateTime AangemaaktDatum { get; set; }
14     public List<Bestelling>? Bestellingen { get; set; }
15 }
16
17 public class OrderLijn
18 {
19     public int Id { get; set; }
20     public double Aantal { get; set; }
21     public int BestellingId { get; set; }
22     public int ProductId { get; set; }
23     public Product? Product { get; set; }
24     public Bestelling? Bestelling { get; set; }
25 }
26
27 public class Product
28 {
29     public int Id { get; set; }
30     public string Naam { get; set; }
31     public string? Beschrijving { get; set; }
32     public decimal Prijs { get; set; }
33     public List<OrderLijn> Orderlijnen { get; set; }
34 }
35
```

Een DTO die enkel de Klant en zijn bestelde producten op een menselijk leesbare manier voorziet zou er als volgt uit kunnen zien:

```
1 public class KlantMetBestellingenDto
2 {
3     public int Id { get; set; }
4     public string KlantNaam { get; set; }
5     public decimal TotaalPrijs { get; set; }
6     public BesteldProductDto[] BesteldeProducten { get; set; } = [];
7 }
```

csharp

```
1 public class BesteldProductDto
2 {
3     public int Id { get; set; }
4     public string Naam { get; set; }
5     public int Aantal { get; set; }
6     public decimal Prijs { get; set; }
7 }
```

csharp

### Waarschuwing

Zoals je hierboven ziet mag een DTO een verwijzing hebben naar een (lijst van) andere Dto(s). Het is uiteraard wel belangrijk dat de andere DTO geen verwijzing mag hebben naar de huidige DTO, want dan zitten we opnieuw met het circulaire probleem.



### Info

We houden DTO's kinderlijk eenvoudig. Het maakt niet uit hoe moeilijk het is om de data achter de schermen te verzamelen; een DTO levert deze kant en klaar, eenvoudig en zo plat (*flat*) mogelijk aan de gebruiker. Dit wil zeggen dat we niet eindeloos objecten binnenin objecten stoppen.

De keuken stelt in dit geval onze Modellen voor: complex, chaotisch en bevat veel meer dan de klant vroeg. Een DTO is het hapklare gerecht op het bord en bevat enkel wat de client nodig heeft.

We gebruiken DTO's overigens ook voor POST en PUT endpoints. Ook daar bevatten ze enkel de properties die we nodig hebben (of toelaten) voor een bepaalde actie.

### Tips

Je kan DTO's vergelijken met ViewModels uit het MVC-patroon, maar dan eenvoudiger en zonder methodes. Beide zijn objecten die specifiek ontworpen zijn voor één bepaalde weergave of actie.

## Waarom DTO's?

Wanneer je in een Web API rechtstreeks EF Core entiteiten (Klant, Bestelling, Product) teruggeeft via je endpoints, ontstaan er vier grote problemen:

### 1. Circulaire Referenties (JSON Lussen)

Zoals eerder gezien: Een Bestelling heeft een Klant, en die Klant heeft op zijn beurt weer een lijst van Bestellingen. Een JSON-serializer raakt hierin verstrikt en blijft oneindig rond-draaien tot de applicatie crasht.

### 2. Overfetching & Privacy (Gevoelige data lekken)

Je entiteiten / modellen bevatten vaak velden die de frontend helemaal niet nodig heeft of zelfs **absoluut niet mag zien**. Denk aan:

- Wachtwoord-hashes, BSN/Rijksregisternummers of interne statuscodes in een Klant model.

- CreatedBy, IsDeleted of database-technische auditvelden. Waarom zou je 20 velden over het netwerk sturen als de mobiele app alleen de Naam en het Email adres nodig heeft?

### 3. Kwetsbaarheid

Stel dat je een `Klant` entiteit rechtstreeks accepteert in een `POST` -actie:

```
1 // ❌ GEVAARLIJK: De gebruiker kan velden meesturen die hij niet mag csharp  
2 aanpassen!  
3 [HttpPost]  
4 public async Task<IActionResult> CreateKlant(Klant klant)  
5 {  
6     _uow.KlantRepository.Add(klant);  
7     await _uow.SaveChangesAsync();  
8     return Ok(klant);  
}
```

Een kwaadwillende gebruiker kan via een tool zoals Postman een JSON-body sturen met een extra veld: "IsAdmin": true of "KredietLimiet": 999999. Als dit veld op jouw entiteit bestaat, zal de Model Binder van C# dit blindelings overnemen. Met DTO's bepaal jij exact welke velden binnen mogen komen.

### 4. Strakke koppeling (Tight Coupling)

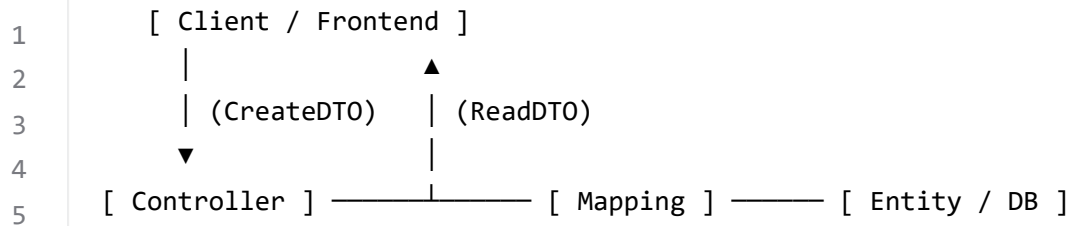
Als je jouw database-tabel hernoemt (bijvoorbeeld `BestelDatum` verandert naar `AangeemaaktOp`), breekt meteen de API voor al jouw frontend-ontwikkelaars en mobiele apps. Een DTO werkt als een buffer/contract tussen jouw database en de buitenwereld.

#### Info

Ga ervan uit dat clients (zoals frontends en apps) jouw DTO's gebruiken als contract en erop vertrouwen dat deze niet zomaar veranderen. Als je datamodel wel verandert, pas je de mapping aan, maar blijft de DTO hetzelfde. Zo hoeven zij hun applicaties niet aan te passen.

## Soorten DTO's: Read vs Write

In de praktijk maken we een strikt onderscheid tussen data die we versturen (Read DTO's) en data die we ontvangen (Create/Update DTO's).



- **Read DTO's** (...ReadDto of ...Dto):

- Bevatten alleen de velden die de client moet zien.
- Dto's worden zoveel mogelijk *platgeslagen*. Dat wil zeggen dat ze zo weinig mogelijk andere modellen bevatten om de leesbaarheid te verhogen.
- Mogen wel Id's bevatten.

- **Write DTO's** (...CreateDto, ...UpdateDto):

- Bevatten enkel de velden die de client mag invullen.
- Geen ID's (die genereert de database of URL).
- Geen berekende velden.

### Info

#### 💡 C# Records vs Classes voor DTO's

In moderne C# (C# 9+) worden DTO's heel vaak geschreven als `record` in plaats van `class`. Records zijn **immutable** (Data is niet aan te passen na aanmaak), lichtgewicht en hebben standaard ingebouwde waarde-vergelijking.

```

1      // Klasse DTO
2      public class KlantDto
3      {
4          public int Id { get; set; }
5          public string Naam { get; set; } = string.Empty;
6          public string Email { get; set; } = string.Empty;
7      }
8
9      // Record DTO (Korter, schoner en immutable)
10     public record KlantDto(int Id, string Naam, string Email);

```

csharp

Beide vormen zijn correct, maar in deze cursus gebruiken we klasse-DTO's met { get; set; } voor maximale duidelijkheid. Je mag uiteraard wel records gebruiken als je dat verkiest.

Verwijder onderstaande lijn die we vorige les aan **Program.cs** hadden toegevoegd. We zullen niet langer gebruik maken van deze dirty fix.

```
1 // Negeer objecten die we al eens geparseerd hebben om oneindige lussen te voorkomen
2 options.JsonSerializerOptions.ReferenceHandler =
  System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;
```

## Get-DTO's aanmaken

Maak een nieuwe folder aan genaamd **DTOs**. Binnen deze map plaats je doorgaans voor elk Model een aparte map waarin je de verschillende DTO's logisch kan groeperen. Voor elk endpoint voorzie je een aparte DTO. De DTO map in dit project zal er als volgt uitzien. Voorzie al de gevraagde bestanden, zodat we deze gaandeweg kunnen invullen.

```
1  JouwAPIProject
2  |  Folder Controllers
3  |  Folder Models
4  |  Folder DTOs
5  |  |  Folder Bestellingen
6  |  |  |  BestellingWriteDto.cs
7  |  |  |  BesteldProductDto.cs
8  |  |  Folder Klanten
9  |  |  |  KlantDto.cs
10 |  |  |  KlantMetBestellingenDto.cs
11 |  |  |  KlantWriteDto.cs
12 |  |  Folder Producten
13 |  |  |  ProductDto.cs
14 |  |  |  ProductWriteDto.cs
15 |  |  Folder OrderLijnen
16 |  |  |  OrderLijnWriteDto.cs
17 |  Folder Configuration
18 |  |  MapperProfile.cs
19 |  Program.cs
```

### Notitie

Let ook op de **Configuration** folder. Hierin plaatsen we later onze automatische Mapping functionaliteit. Hierover later meer in dit hoofdstuk.

Laten we beginnen met de DTO's om data aan de client te tonen. Het leuke aan GET-DTO's is dat we deze zo eenvoudig mogelijk kunnen opstellen. Het is een simpele klasse (of record) die enkel de data bevat die op een specifiek endpoint getoond moet worden. Je

hoeft geen rekening te houden met complexe modelstructuren. Het zijn puur eenvoudige klassen met uitsluitend de benodigde properties.

### Tips

Leg je Modellen naast je DTO's terwijl je deze ontwikkelt en probeer de propertynamen zo veel mogelijk exact over te nemen. Dit bespaart je later veel werk wanneer we de data gaan overzetten (mappen).

## ProductDto.cs

Laten we beginnen met de ProductDto. Zoals gezegd, plaatsen we in een DTO **enkel de velden die we aan de client willen tonen**. Als we productinformatie opvragen, zijn we via dit endpoint niet geïnteresseerd in alle individuele orderlijnen waarin dit product ooit is verkocht. We kunnen het originele Model dus grotendeels overnemen, maar we laten de onderliggende relaties (zoals de lijst van OrderLijnen) gewoon weg.

### Class

```
1 public class ProductDto
2 {
3     public int Id { get; set; }
4     public string Naam { get; set; }
5     public string? Beschrijving { get; set; }
6     public decimal Prijs { get; set; }
7 }
```

csharp

### Record

```
1 public record ProductDto(int Id, string Naam, string? Beschrijving,
    decimal Prijs);
```

csharp

## KlantMetBestellingenDto

Voor de KlantMetBestellingenDto kunnen we selectiever te werk gaan. Zoals eerder in dit hoofdstuk beschreven, hebben we een groot aantal Modellen nodig om alle Klant - Bestelling - OrderLijn - Product informatie op te halen. Voor een DTO zal dit ons echter worst we-

zen. We kijken enkel naar de properties die we effectief willen tonen, zoals de samengestelde klantnaam (voornaam en naam), de totale prijs (die we later berekenen) en de producten die in de bestelling zaten.

De lijst van bestelde producten kan een uitdaging zijn. We willen namelijk meer tonen dan alleen een lijst van productnamen; we willen bijvoorbeeld ook de prijs per product en het aantal bestelde items zien. De verleiding is dan snel groot om simpelweg een verwijzing naar het Product-model eraan toe te voegen. Doe dit **NIET**. Modellen en DTO's moeten strikt gescheiden blijven. We maken hiervoor gewoon een nieuwe DTO aan en stoppen daar de gewenste informatie in.

### Waarschuwing

Om de complexiteit te beperken en problemen met circulaire referenties te vermijden, is het uiterst belangrijk dat DTO's NOOIT Modellen (Entiteiten) bevatten en vice versa. Een DTO mag wel (een lijst van) andere DTO's bevatten, maar Modellen en DTO's horen strikt gescheiden te blijven.

#### Class

```
1 public class KlantMetBestellingenDto
2 {
3     public int Id { get; set; }
4     public string KlantNaam { get; set; }
5     public decimal TotaalPrijs { get; set; }
6     public BesteldProductDto[] BesteldeProducten { get; set; } = [];
7 }
```

csharp

#### Record

```
1 public record KlantMetBestellingenDto(int Id, string KlantNaam, decimal
   TotaalPrijs, BesteldProductDto[] BesteldeProducten);
```

csharp

## KlantDto.cs

#### Class

```
1 namespace WebApiDemo.DTOs.Klanten;
2
3 public class KlantDto
4 {
5     public int Id { get; set; }
6     public string Naam { get; set; }
7     public string Voornaam { get; set; }
8     public DateTime AangemaaktDatum { get; set; }
9
10    public BesteldProductDto[] BesteldeProducten{ get; set; }
11 }
```

csharp

### Record

```
1 public record KlantDto(int Id, string Naam, string? Voornaam, DateTime AangemaaktDatum, BesteldProductDto[] BesteldeProducten);
```

csharp

## BesteldProductDto

We doen hetzelfde voor de BesteldProductDto. De data en relaties in de onderliggende Modellen kunnen relatief complex zijn, maar dat hoeft niet zo te zijn voor onze DTO. Deze houden we heel eenvoudig met primitieve datatypes zoals int, string en decimal. Het maakt voor de DTO ook niet uit waar de data exact vandaan komt. Zolang er een Model is dat de benodigde data bevat, is het eenvoudig om de property later op te vullen (mappen).

### Info

Merk op dat KlantMetBestellingenDto een verwijzing heeft naar BesteldProductDto, maar omgekeerd niet. Anders introduceren we opnieuw het circulaire probleem uit het vorige hoofdstuk, en dat is nu net één van de problemen die we met DTO's trachten op te lossen.

### Class

```
1 public class BesteldProductDto
2 {
3     public int Id { get; set; }
4     public string Naam { get; set; }
5     public int Aantal { get; set; }
6     public decimal Prijs { get; set; }
7 }
```

csharp

## Record

```
1 public record BesteldProductDto(int Id, string Naam, int Aantal,
    decimal Prijs);
```

csharp

# Mapping

## Het concept van Mapping

Nu we de eerste GET-DTO's hebben aangemaakt, kunnen we stoppen met het rechtstreeks blootstellen van onze interne databasemodellen aan de buitenwereld. Voordat we deze DTO's via onze API kunnen terugsturen, moeten we ze natuurlijk wel eerst vullen met de juiste data uit onze originele Modellen.

Dit proces noemen we **Mapperen** : het overzetten of transformeren van data uit object A (het Model) naar object B (de DTO) of vice versa.

## Nog meer gedupliceerde code en complexiteit?

Telkens we DTO's introduceren in de les, krijgen we vaak de volgende kritiek van studenten en beginnende ontwikkelaars:

*Maar nu moet ik voor bijna elke tabel in de database nóg minstens twee extra klassen (een GetDto en een WriteDto) aanmaken. Bovendien moet ik de data van het ene object handmatig naar het andere kopiëren! Dit is toch gewoon dubbel werk?*

Het klopt inderdaad dat je codebase groeit en dat je meer klassen moet schrijven. Maar deze strikte scheiding ( *Separation of Concerns* ) voorkomt enorme hoofdpijn als je applicatie groter wordt. Je voorkomt dat een database-wijziging direct resulteert in crashende

front-ends, en je dekt veiligheidsrisico's (zoals over-posting) definitief af. De extra tijd die je steekt in het typen van deze klassen, weegt ruimschoots op tegen de tijd die je bespaart met urenlang debuggen.

Daarnaast valt de hoeveelheid extra "kopieerwerk" enorm mee. We gaan dit namelijk niet handmatig doen. We maken gebruik van een gespecialiseerde Mapping Library om de overzettingen van Model naar DTO (en vice versa) volledig te automatiseren.

Er zijn verschillende Packages beschikbaar waarin al het zware overzetwerk al geschreven werd zoals AutoMapper, Mapster, Mapperly... In deze cursus maken we gebruik van Mapster, maar je mag ook een ander framework gebruiken. Het belangrijkste is dat je de concepten begrijpt. Syntax kan je altijd opzoeken.

## Handmatig Mappen (Hoe we het NIET willen doen)

Als we geen bibliotheek zouden gebruiken, zou onze code in een Controller er als volgt uitzien:

```
1 // 1. Data ophalen
2 Product product = _context.Producten.Find(1);
3
4 // 2. Handmatig mappen (Monnikenwerk!)
5 ProductDto productDto = new ProductDto()
6 {
7     Id = product.Id,
8     Naam = product.Naam,
9     Beschrijving = product.Beschrijving,
10    Prijs = product.Prijs
11 };
12
13 // 3. Data terugsturen
14 return Ok(productDto);
```

csharp

Beeld je in dat je dit moet doen voor Modellen met 20 properties, of voor hele lijsten van objecten. Dit is foutgevoelig, saai en overbodig werk. Ook zou je alle plaatsen waar dit soort code voorkomt moeten aanpassen iedere keer er een verandering in een Model plaatsvindt. Dit is vragen voor problemen in de toekomst.

Gelukkig is er een betere manier. Er bestaan in C# verschillende "Mappers" (zoals AutoMapper en Mapperly), maar wij kiezen voor Mapster. Mapster is bliksemsnel, vereist nageenog geen configuratie en leest heel natuurlijk.

## Automatisch mappen met Mapster

Er bestaan in C# verschillende "Mappers" (zoals AutoMapper en Mapperly), maar wij kiezen voor Mapster. Mapster is bliksemsnel, vereist nagenoeg geen configuratie en leest heel natuurlijk.

### Mapster Installeren

Mapster is een NuGet package. Voeg deze toe aan je Web API project via de NuGet Package Manager, of via de terminal:

1

```
dotnet add package Mapster
```

bash

## Controllers aanpassen naar DTO's

---

Laten we eenvoudig beginnen met ProductController. De overstap naar DTO's in je controller is met Mapster verrassend eenvoudig. Zolang de namen van de properties in je domeinmodel (Product) en je DTO (ProductDto) overeenkomen, doet Mapster het zware tilwerk via zogenaamde convention-based mapping.

### Hoe werkt Mapster?

Mapster is slim. Het kijkt naar de property-namen van je Model en vergelijkt deze met de property-namen van je DTO. Als de namen en datatypes overeenkomen (bijvoorbeeld de Prijs in Product en de Prijs in ProductDto), dan kopieert Mapster de waarde automatisch over.

Dit doet Mapster via een handige zogenaamde Extension Method genaamd `.Adapt<T>()`. Je hoeft bovenaan je bestand enkel `using Mapster;` toe te voegen.

## ProductController.cs

```
1  using Mapster;
2  using WebApiDemo.DTOs.Producten;
3
4  namespace WebApiDemo.Controllers
5  {
6      [ApiController]
7      [Route("api/[controller]")]
8      public class ProductController : ControllerBase
9      {
10         // Onze "ingrediënten" (Dependencies)
11         private readonly IUnitOfWork _uow;
12         private readonly ILogger<ProductController> _logger;
13
14         // Vraag om en injecteer Dependencies
15         public ProductController(IUnitOfWork uow,
16         ILogger<ProductController> logger)
17         {
18             _uow = uow;
19             _logger = logger;
20         }
21
22         [HttpGet("{id}")]
23         public async Task<ActionResult<ProductDto>> GetProduct(int id)
24         {
25             _logger.LogInformation($"GET request voor Product met ID:
26             {id}");
27
28             Product? product = await
29             _uow.ProductRepository.GetByIdAsync(id);
30
31             if (product == null)
32             {
33                 _logger.LogWarning($"Product met ID {id} werd niet
34             gevonden.");
35                 return NotFound($"Helaas, we konden geen product vinden
36             met Id {id}.");
37             }
```

csharp

```
38         // Map Model naar DTO
39         ProductDto productDto = product.Adapt<ProductDto>();
40
41         return Ok(productDto);
42     }
43
44     [HttpGet()]
45     public async Task<ActionResult<ProductDto[]>> GetAllProducts(int
46 id)
47     {
48         _logger.LogInformation($"GET request voor alle Producten");
49
50         IEnumerable<Product> products = await
51 _uow.ProductRepository.GetAllAsync();
52
53         if (products == null || !products.Any())
54         {
55             _logger.LogWarning("Geen producten gevonden.");
56             return NotFound("Geen producten gevonden.");
57         }
58
59         // Map collectie van Modellen naar collectie van DTO's
60         ProductDto[] productDtos = products.Adapt<ProductDto[]>();
61
62         return Ok(productDtos);
63     }
64 }
```

### Tips

Waarom heten onze properties hetzelfde? Weet je nog de tip uit het begin van dit hoofdstuk? *Leg de Modellen naast je DTO's en neem de propertynamen exact over.* Hier zie je exact waarom dat cruciaal is. Doordat wij onze properties exact dezelfde naam en type hebben gegeven (bijv. Beschrijving en niet ProductBeschrijving), hoeven we Mapster absoluut niets aan te leren. Het werkt out-of-the-box!

Wat verandert er?

- **De Returnwaarde:** Je methode geeft nu een `Task<ActionResult<ProductDto>>` terug in plaats van het ruwe domeinmodel.

- **De Transformatie:** Je haalt nog steeds het originele model op uit de repository, maar voordat je het terugstuurt, roep je `.Adapt<ProductDto>()` aan.
- **De Output:** Je plaatst de resulterende DTO in de `Ok()` response, zodat de client alleen de veilige, gefilterde data ontvangt.

The top screenshot shows a REST client interface with a GET request to `https://localhost:7863 /api/Product/{id}`. The response is a 200 OK status with a body containing a single product DTO:

```

1 {
2   "id": 1,
3   "naam": "fiets",
4   "beschrijving": "Dit is een fiets",
5   "prijs": 100.00
6 }

```

The bottom screenshot shows a REST client interface with a GET request to `https://localhost:7863 /api/Product`. The response is a 200 OK status with a body containing a list of three product DTOs:

```

1 [
2   {
3     "id": 1,
4     "naam": "fiets",
5     "beschrijving": "Dit is een fiets",
6     "prijs": 100.00
7   },
8   {
9     "id": 2,
10    "naam": "koersfiets",
11    "beschrijving": "Dit is een mooie koersfiets",
12    "prijs": 200.00
13  },
14  {
15    "id": 3,
16    "naam": "auto",
17    "beschrijving": "Dit is een auto",
18    "prijs": 2000.00
19  }
20 ]

```

### Notitie

Complexer dan dit hoeft mapping niet te worden binnen Controllers. Het is slechts een kwestie van je returntype aanpassen en de mapping toevoegen. Bang voor te veel bestanden en extra werk? Nergens voor nodig. Zolang je alles netjes in mappen indeelt, betekent meer bestanden absoluut geen chaos, maar juist strak georganiseerde code.

## BestellingController.cs

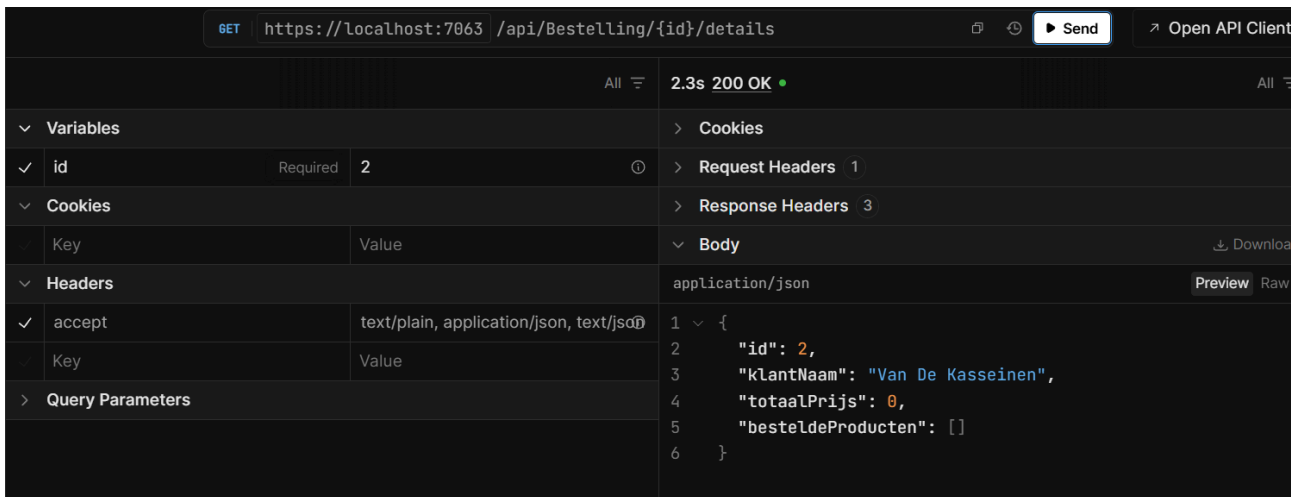
Pas nu op dezelfde manier de BestellingController aan, zodat de endpoints ook hier DTO's returnen. Let er ook dat de Controllers de correcte Repository methodes oproepen die met Include() gerelateerde data ophalen.

```
1  using Mapster;
2  using WebApiDemo.DTOs.Klanten;
3
4  namespace WebAPIDemo.Controllers
5  {
6      [Route("api/[controller]")]
7      [ApiController]
8      public class BestellingController : ControllerBase
9      {
10         // 1. We gebruiken enkel nog IUnitOfWork, geen losse repositories
11         meer!
12         private readonly IUnitOfWork _uow;
13
14         public BestellingController(IUnitOfWork uow)
15         {
16             _uow = uow;
17         }
18
19         // GET: api/Bestellingen/5/details
20         // Haalt een bestelling op inclusief Klant, OrderLijnen en
21         Producten
22         [HttpGet("{id}/details")]
23         public async Task<ActionResult<KlantMetBestellingenDto>>
24         GetBestellingMetDetails(int id)
25         {
26             // Gebruik de specifieke repository-methode via de Uow
27             Bestelling? bestelling = await
28             _uow.BestellingRepository.GetBestellingMetDetailsAsync(id);
29
30             if (bestelling == null)
31             {
32                 return NotFound();
33             }
34         }
35     }
```

csharp

```
34         // Zet het model om naar een DTO
35         KlantMetBestellingenDto dto =
36         bestelling.Adapt<KlantMetBestellingenDto>();
37
38         // Retourneer de DTO
39         return Ok(dto);
40     }
41
42     // GET: api/Bestellingen/klant/3
43     // Haalt een klant op inclusief al zijn bestellingen en
44     gekoppelde producten
45     [HttpGet("klant/{klantId}")]
46     public async Task<ActionResult<Klant>>
47     GetBestellingenVanKlant(int klantId)
48     {
49         // Hier spreken we de KlantRepository aan via dezelfde Uow
50         Klant? klant = await
51         _uow.KlantRepository.GetKlantMetBestellingen(klantId);
52
53         if (klant == null)
54         {
55             return NotFound();
56         }
57
58         // Zet het model om naar een DTO
59         KlantDto dto = klant.Adapt<KlantDto>();
60
61         return Ok(dto);
62     }
```

Na het testen van deze endpoints zal je merken dat grote delen van je DTO ontbreken (NULL of leeg zijn).



GET `https://localhost:7063 /api/Bestelling/{id}/details` Send Open API Client

2.3s 200 OK

Variables: id (Required, 2)

Cookies

Headers: accept (text/plain, application/json, text/javascript)

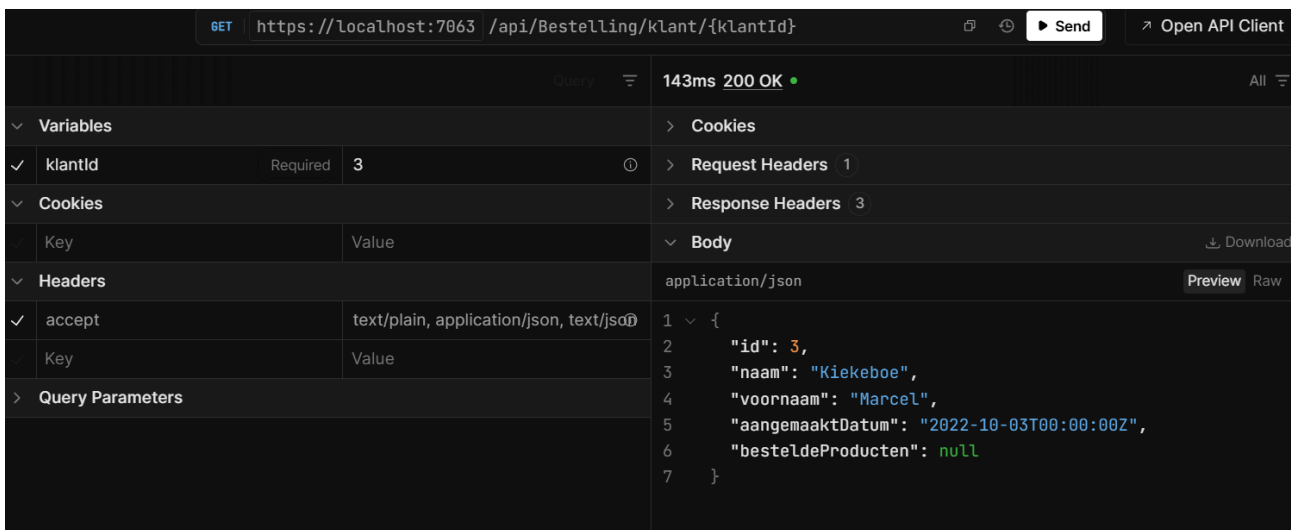
Query Parameters

Request Headers: 1

Response Headers: 3

Body: application/json

```
1 {  
2   "id": 2,  
3   "klantNaam": "Van De Kasseinen",  
4   "totaalPrijs": 0,  
5   "besteldeProducten": []  
6 }
```



GET `https://localhost:7063 /api/Bestelling/klant/{klantId}` Send Open API Client

143ms 200 OK

Variables: klantId (Required, 3)

Cookies

Headers: accept (text/plain, application/json, text/javascript)

Query Parameters

Request Headers: 1

Response Headers: 3

Body: application/json

```
1 {  
2   "id": 3,  
3   "naam": "Kiekeboe",  
4   "voornaam": "Marcel",  
5   "aangemaaktDatum": "2022-10-03T00:00:00Z",  
6   "besteldeProducten": null  
7 }
```

Dat is omdat Mapster niet genoeg informatie heeft waar het deze data kan vinden. Om ook deze data op te vullen zullen we aan Mapster custom Mappings moeten toevoegen.

## Custom Mapping configureren

Soms komen de properties van je Model en DTO niet perfect overeen. Misschien wil je de voor- en achternaam samenvoegen, een totale prijs berekenen op basis van onderliggende tabellen, of ontbreken bepaalde velden.

In Mapster los je dit op door eenmalig een Configuratie aan te maken waarin je de uitzonderingen definieert. Alles wat wel overeenkomt, blijft Mapster automatisch voor je doen.

Om Mapster te vertellen hoe specifieke velden gemapt moeten worden, gebruiken we `TypeAdapterConfig`. Je stelt dit doorgaans één keer in bij het opstarten van je applicatie (bijvoorbeeld in `Program.cs` of een aparte configuratieklasse).

We gaan twee configuraties schrijven: één om een OrderLijn te mappen naar een Besteld-ProductDto, en één om een Bestelling te mappen naar een KlantMetBestellingenDto.

## MapperProfile.cs

Maak een nieuwe klasse **MapperProfile** aan in de map Configuration. Hierin plaats je alle specifieke mappings. Properties uit het model die exact **dezelfde naam, hetzelfde type en dezelfde structuur** (root location) hebben, worden automatisch overgezet en hoef je niet handmatig te definiëren.

Afwijkende properties map je handmatig door eerst `config.NewConfig<SourceObject, DestinationObject>()` aan te maken. Daarna chain je de methode `.Map(dest => destinationProperty, src => sourceProperty)`. Je kunt deze methodes eindeloos achter elkaar aanroepen (*chainen*). Het is hierbij zelfs mogelijk om een combinatie van bronproperties samen te voegen voor één doelproperty.

```
1  using Mapster;
2  using WebApiDemo.DTOs.Klanten;
3
4  namespace WebApiDemo.Configuration;
5
6  public class MapperProfile : IRegister
7  {
8      public void Register(TypeAdapterConfig config)
9      {
10         config.NewConfig<Bestelling, KlantMetBestellingenDto>()
11             // Namen samenvoegen
12             .Map(dest => dest.KlantNaam, src => $"{src.Klant.Voornaam}
13 {src.Klant.Naam}")
14
15             // Totale prijs berekenen door alle orderlijnen te overlopen.
16             (Bad Practice)
17             .Map(dest => dest.TotaalPrijs, src => src.OrderLijnen.Sum(ol
18 => (decimal)ol.Aantal * ol.Product!.Prijs))
19
20             // Relatie mappen (Mapster snapt automatisch dat hij hiervoor
21 de bovenstaande regel 1 moet gebruiken)
22             .Map(dest => dest.BesteldeProducten, src => src.OrderLijnen);
23
24             // De namen van deze Properties zijn hetzelfde, maar zitten een
25 niveau dieper in de Source.
26             // Daarom moeten we deze manueel mappen
```

```
26         config.NewConfig<OrderLijn, BesteldProductDto>()
27             .Map(dest => dest.Naam, src => src.Product!.Naam)
28             .Map(dest => dest.Prijs, src => src.Product!.Prijs);
29
30         //Geavanceerdere mappings
31
32         // 1. Vertel Mapster hoe één OrderLijn naar één BesteldProductDto
33         vertaald moet worden
34         config.NewConfig<OrderLijn, BesteldProductDto>()
35             .Map(dest => dest.Id, src => src.Product.Id)
36             .Map(dest => dest.Aantal, src => src.Aantal)
37             .Map(dest => dest.Naam, src => src.Product.Naam)
38             .Map(dest => dest.Prijs, src => src.Product.Prijs);
39
40         // 2. Vertel Mapster hoe de Klant naar de KlantDto vertaald moet
41         worden
42         config.NewConfig<Klant, KlantDto>()
43             // Id, Naam, Voornaam en AangemaaktDatum worden automatisch
44             gemapt!
45             // Gebruik SelectMany om alle OrderLijnen uit alle
46             Bestellingen samen te voegen tot één platte lijst
47             .Map(dest => dest.BesteldeProducten, src =>
48                 src.Bestellingen.SelectMany(b => b.OrderLijnen));
49     }
50 }
```

### Info

De MapperProfile-klasse fungeert als het centrale regelboek voor Mapster, waarin je definieert hoe complexe datamodellen veilig worden omgezet naar vereenvoudigde DTO's. Dit houdt je code overzichtelijk en haalt complexe transformaties uit je Controllers.

Kernconcepten van deze configuratie:

- **Automatische mapping:** Eigenschappen met exact dezelfde naam en datastructuur worden onzichtbaar en automatisch overgezet.
- **Handmatige mapping (.Map):** Wordt gebruikt om afwijkende structuren te vertalen, zoals het ophalen van diep geneste data (bijv. de naam uit een onderliggend Product-object) of het combineren van velden (voornaam en achternaam).
- **Lijsten platslaan:** Met LINQ-commando's zoals .SelectMany() transformeer je complexe hiërarchieën (zoals een klant met meerdere bestellingen, die elk weer orderlijnen bevatten) moeiteloos naar één platte, leesbare lijst voor de frontend.

### Tips

Hoewel je tijdens het mappen berekeningen kunt uitvoeren, wordt dit sterk afgeraden:

- **Slechte testbaarheid:** Code binnen een MapperProfile is lastig te debuggen en te unittesten.
- **Verborgene logica:** Voorkom dat het profiel een ondoorzichtige magic-happens-here-klasse wordt.
- **Scheiding van verantwoordelijkheden:** Beperk mapping-code puur tot het overzetten van data. Voer complexe logica of berekeningen altijd uit in Controllers, Services of Helper-klassen.

## Mapster activeren in Program.cs

Om ervoor te zorgen dat je applicatie deze configuratie-klasse effectief leest wanneer de API opstart, moet je één regel code toevoegen aan je Program.cs.

Voeg deze regel toe ergens bovenaan in Program.cs (bijvoorbeeld net na `var builder = WebApplication.CreateBuilder(args)`): 😊

```
1 using Mapster;
2 using System.Reflection;
3 using Scalar.AspNetCore;
4
5 var builder = WebApplication.CreateBuilder(args);
6
7 // Vertel Mapster om het huidige project te scannen op IRegister klassen
8 // (zoals onze MapperProfile)
9 TypeAdapterConfig.GlobalSettings.Scan(Assembly.GetExecutingAssembly());
10
11 // ... rest van je Program.cs (Controllers toevoegen, Swagger, DbContext,
12 // etc.)
```

### Tips

Zodra deze opstelling (Setup) eenmalig in Program.cs en MapperProfile.cs is gedaan, blijft de code in de Controllers extreem schoon. Je hoeft bij complexe mappings enkel een nieuwe .Map(...) regel toe te voegen aan het profiel, en kan in de Controller gewoon blijven vertrouwen op bestelling.Adapt<KlantMetBestellingenDto>();

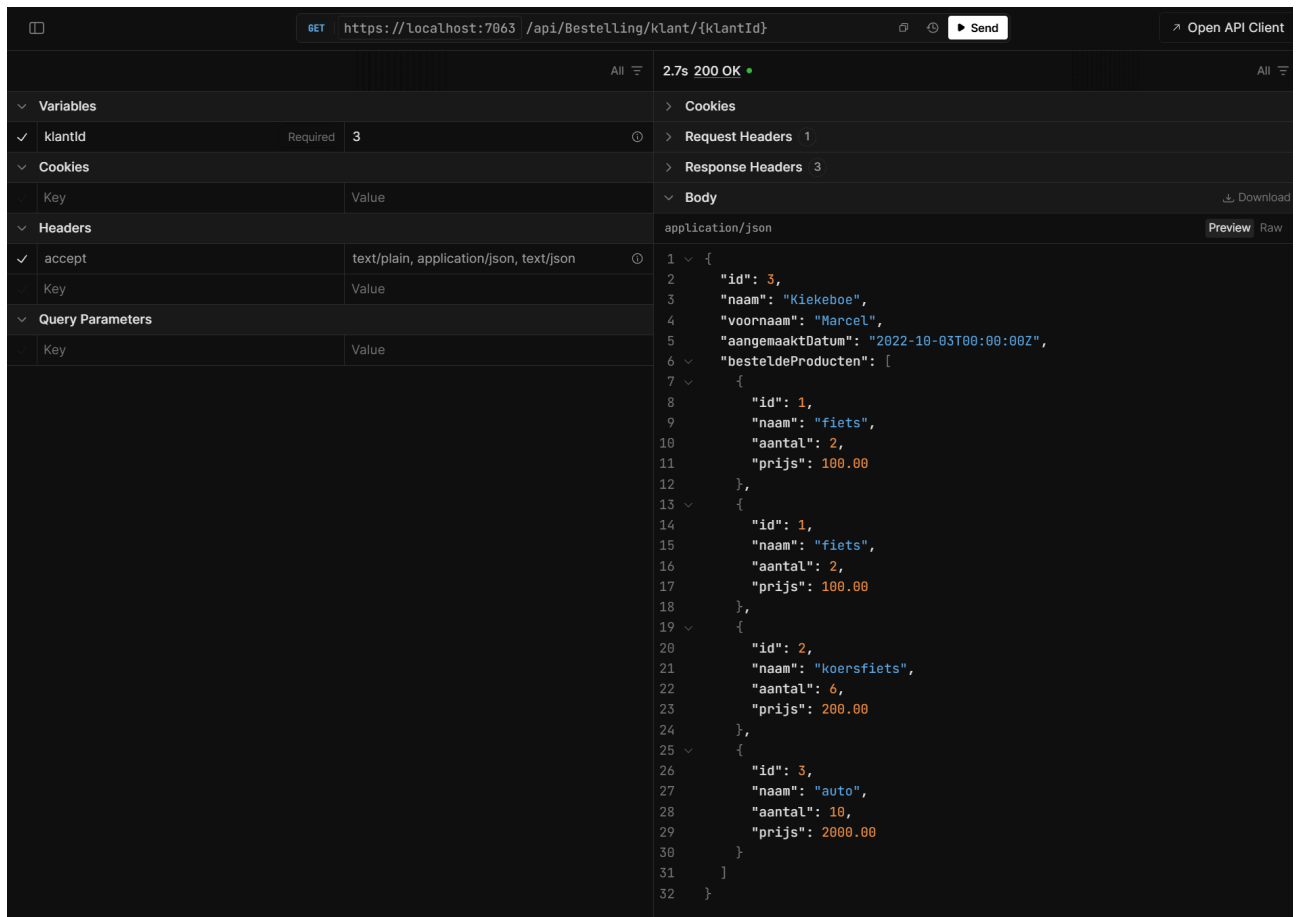
## Resultaat testen

Nu onze Impliciete en expliciete Mappings voorzien zijn, zouden opnieuw alle Get end-points correct moeten werken en foutloze data tonen, ditmaal beperkt tot enkel wat we willen zien.

The screenshot shows an API client interface with the following details:

- URL:** `https://localhost:7063 /api/Bestelling/{id}/details`
- Method:** GET
- Status:** 200 OK
- Response Headers:** application/json
- Body:** A JSON object representing an order and its items.

```
{
  "id": 2,
  "klantNaam": "Firmin Van De Kasseinen",
  "totaalPrijs": 600.00,
  "besteldeProducten": [
    {
      "id": 1,
      "naam": "fiets",
      "aantal": 4,
      "prijs": 100.00
    },
    {
      "id": 2,
      "naam": "koersfiets",
      "aantal": 1,
      "prijs": 200.00
    }
  ]
}
```



## Samenvatting

- **DTO (Data Transfer Object)** : Een afgeslankte klasse bedoeld voor communicatie met de client, waardoor je problemen zoals circulaire referenties, privacy-lekken en overfetting oplost.
- **Read vs Write** : We maken aparte DTO's voor het verzenden van data (GET) en het ontvangen van data (POST/PUT).
- **Geen Modellen in DTO's** : Een Model/Entiteit mag nooit in een DTO voorkomen.
- **Data Annotations** : Worden toegepast op Write-DTO's om gebruikersinput direct en automatisch te valideren.
- **Mapster** : Automatiseert het omzetten ("mappen") van data tussen Modellen en DTO's, zodat je controllers netjes en overzichtelijk blijven.

**Laatst bijgewerkt:**09-09-2026, 08:52

**Bijdragers:** JorenSynaeveTM